SIMATS ENGINEERING

ASSESSMENT TOOL 2

Course: Database Management Systems (CSA05)

Course Outcome: CO3 — Functional dependencies, normalization (1NF–5NF), key constraints

Activity Tool: Case Study (Medium) | Weightage: 30%

Total Marks: 50 (10 × 5 marks)

Code of Conduct

I **S. Lakshmi Priya** (Name) | Reg. No: **192524284** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: S. Lakshmi Priya

> **Q1.** *Analyze the case: An online shopping platform stores Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price). Identify FDs and normalize to 3NF.*

Functional Dependencies

- CustID → CustName
- ProdID → ProdName, Price
- OrderID → CustID, ProdID, Qty

Analysis

CustName depends on CustID, and CustID depends on OrderID — this is a transitive dependency (OrderID → CustID → CustName), the same pattern applies to ProdID → ProdName/Price. Both violate 3NF.

3NF Decomposition

`CUSTOMER(CustID, CustName)`

`PRODUCT(ProdID, ProdName, Price)`

`ORDERS(OrderID, CustID, ProdID, Qty)`

Every non-key attribute now depends directly and only on the key of its own relation; the rejoin of ORDERS ⋈ CUSTOMER ⋈ PRODUCT reconstructs the original data exactly.

Executed Output (MySQL)

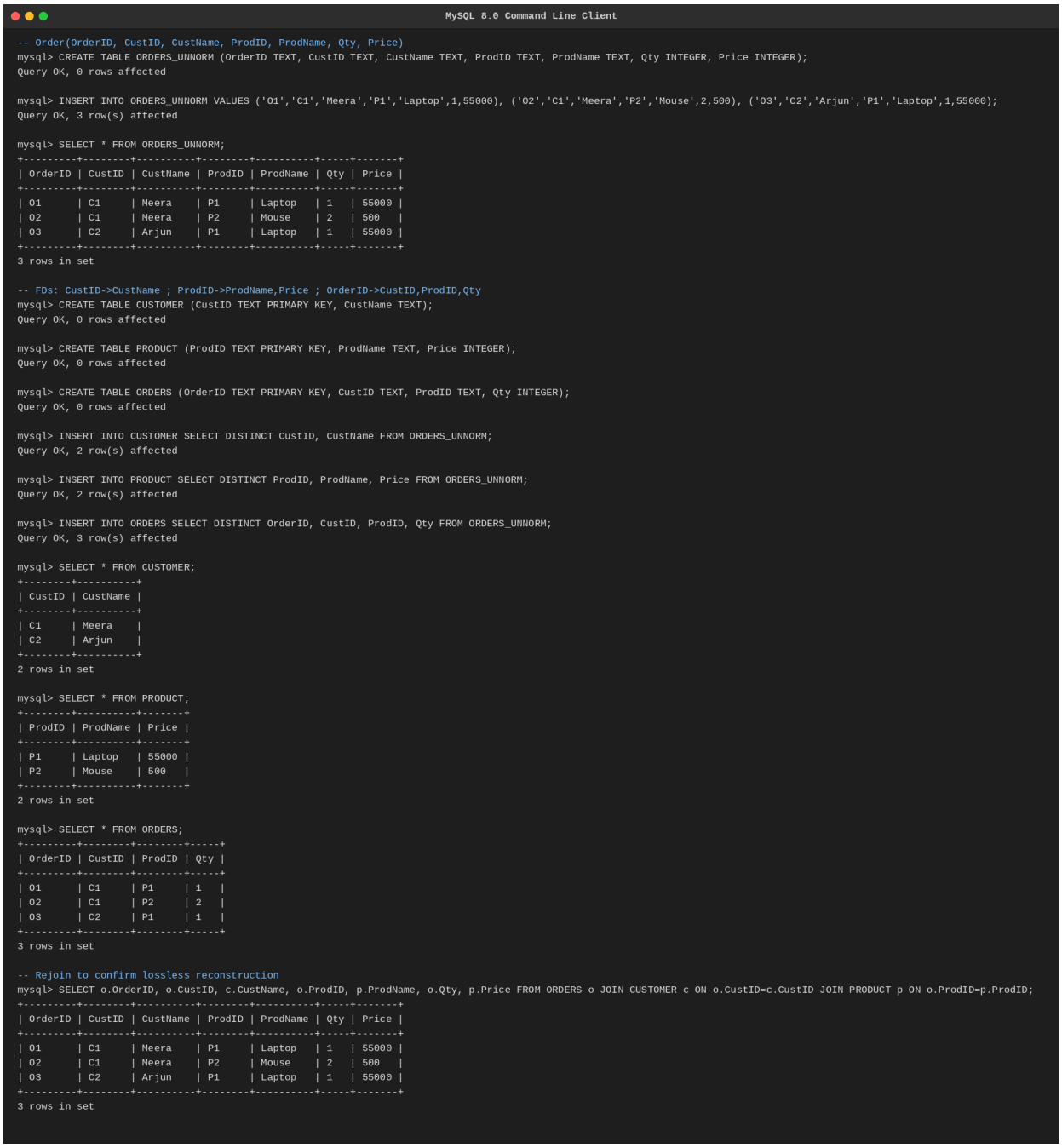

Fig. Q1 — 3NF decomposition executed in MySQL.

> **Q2.** *A hospital maintains Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName). Identify anomalies and normalize to BCNF.*

Functional Dependencies

- PID → PName, DocID, WardNo
- DocID → DocName, Specialization
- WardNo → WardName

Anomalies in the unnormalized form

- Insertion: a new doctor cannot be added until a patient is assigned to them.

- Deletion: discharging the only patient under a doctor erases that doctor's specialization record.
- Update: a doctor's specialization change must be repeated across every patient row for that doctor.

BCNF Decomposition

`DOCTOR(DocID, DocName, Specialization)`

`WARD(WardNo, WardName)`

`PATIENT(PID, PName, DocID, WardNo)`

Every determinant (PID, DocID, WardNo) is now the primary key of its own relation, satisfying BCNF, and each anomaly above is eliminated.

Executed Output (MySQL)

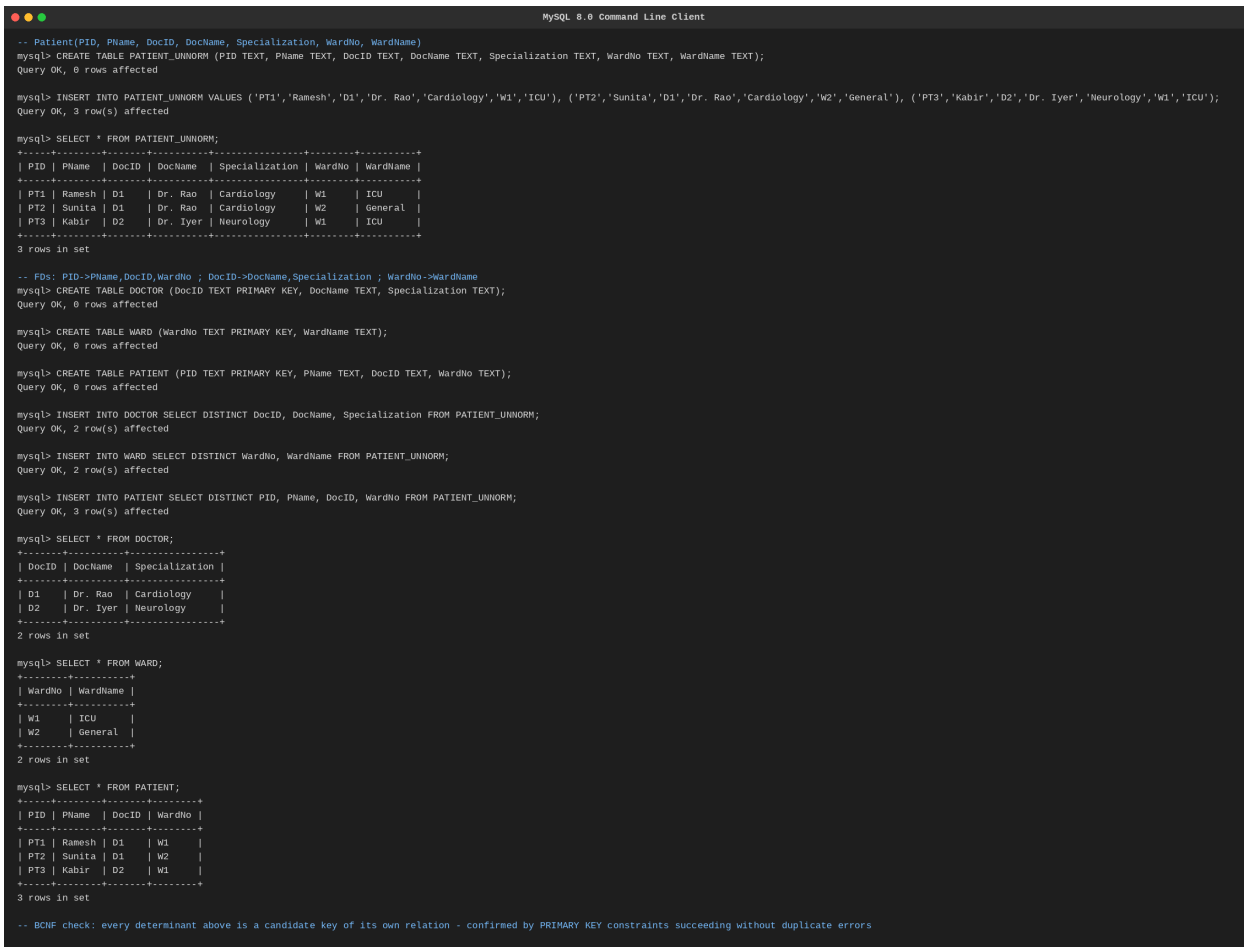

Fig. Q2 — BCNF decomposition executed in MySQL.

> **Q3.** *Case study: A university stores Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade). Analyze FDs and apply 2NF decomposition.*

Functional Dependencies

- SID → SName
- CourseID → CourseName, Faculty
- SID, CourseID → Grade

Candidate key and partial dependencies

The candidate key is the composite {SID, CourseID}. SName depends only on SID (part of the key), and CourseName/Faculty depend only on CourseID (also part of the key) — both are partial dependencies, violating 2NF.

2NF Decomposition

`STUDENT(SID, SName)`

`COURSE(CourseID, CourseName, Faculty)`

`ENROLL(SID, CourseID, Grade)`

Grade — the only attribute that truly needs the full composite key — now sits alone with {SID, CourseID} in ENROLL, removing the partial dependencies.

Executed Output (MySQL)

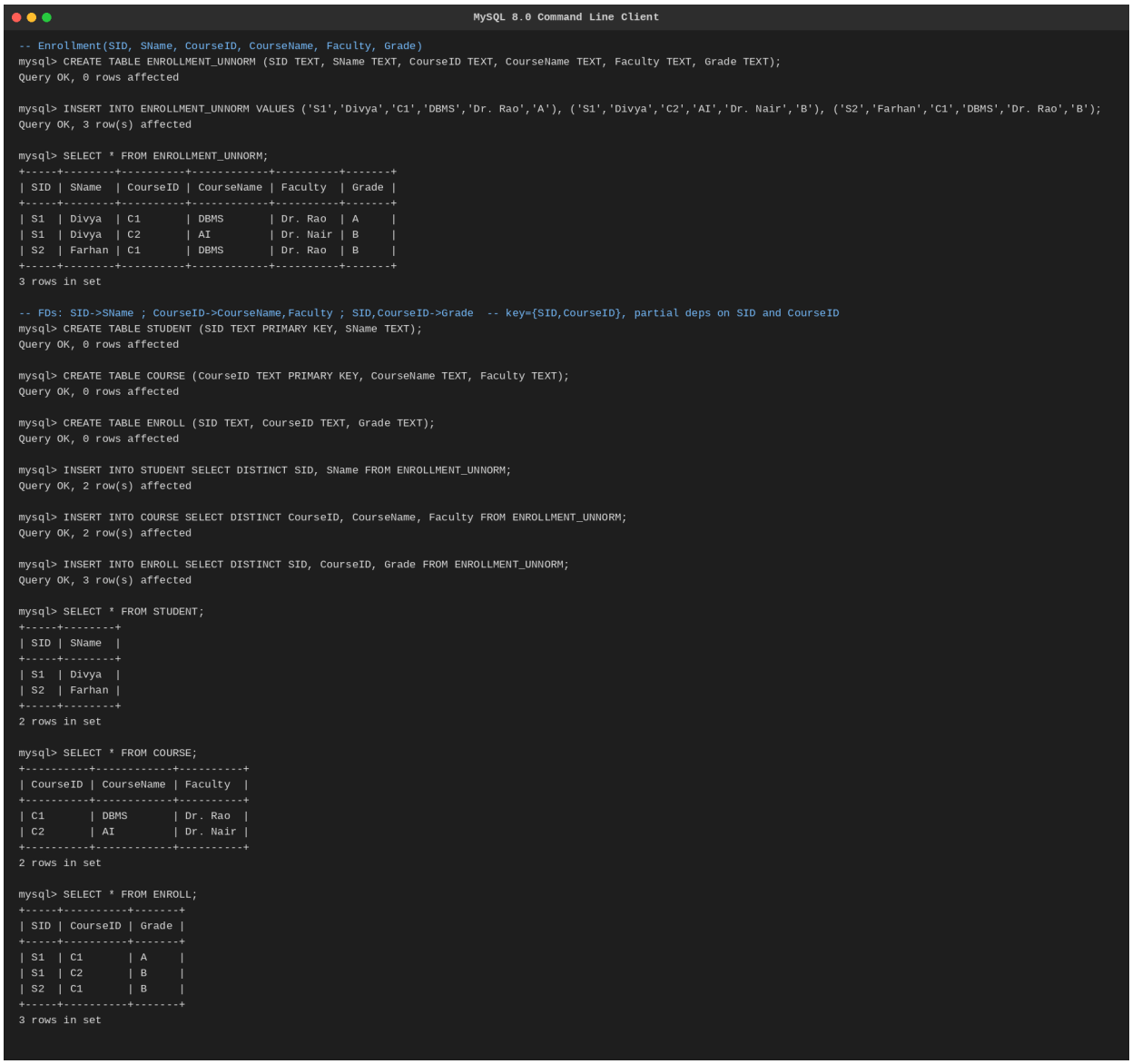

Fig. Q3 — 2NF decomposition executed in MySQL.

Q4. *Given the airline booking case: Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date).*

Normalize up to BCNF.

Functional Dependencies

- FlightNo → DeptCity, ArrCity
- PassID → PassName
- FlightNo, Date, Seat → PassID (a seat on a flight on a given date is held by exactly one passenger)

Candidate key

{FlightNo, Date, Seat} is the candidate key of the booking fact. FlightNo→DeptCity/ArrCity and PassID→PassName are both determinants that are not superkeys of the whole relation, violating BCNF.

BCNF Decomposition

`FLIGHT(FlightNo, DeptCity, ArrCity)`

`PASSENGER(PassID, PassName)`

`BOOKING(FlightNo, Date, Seat, PassID)`

Every determinant is now a key of its own relation; the composite key of BOOKING correctly models one passenger per seat per flight per date.

Executed Output (MySQL)

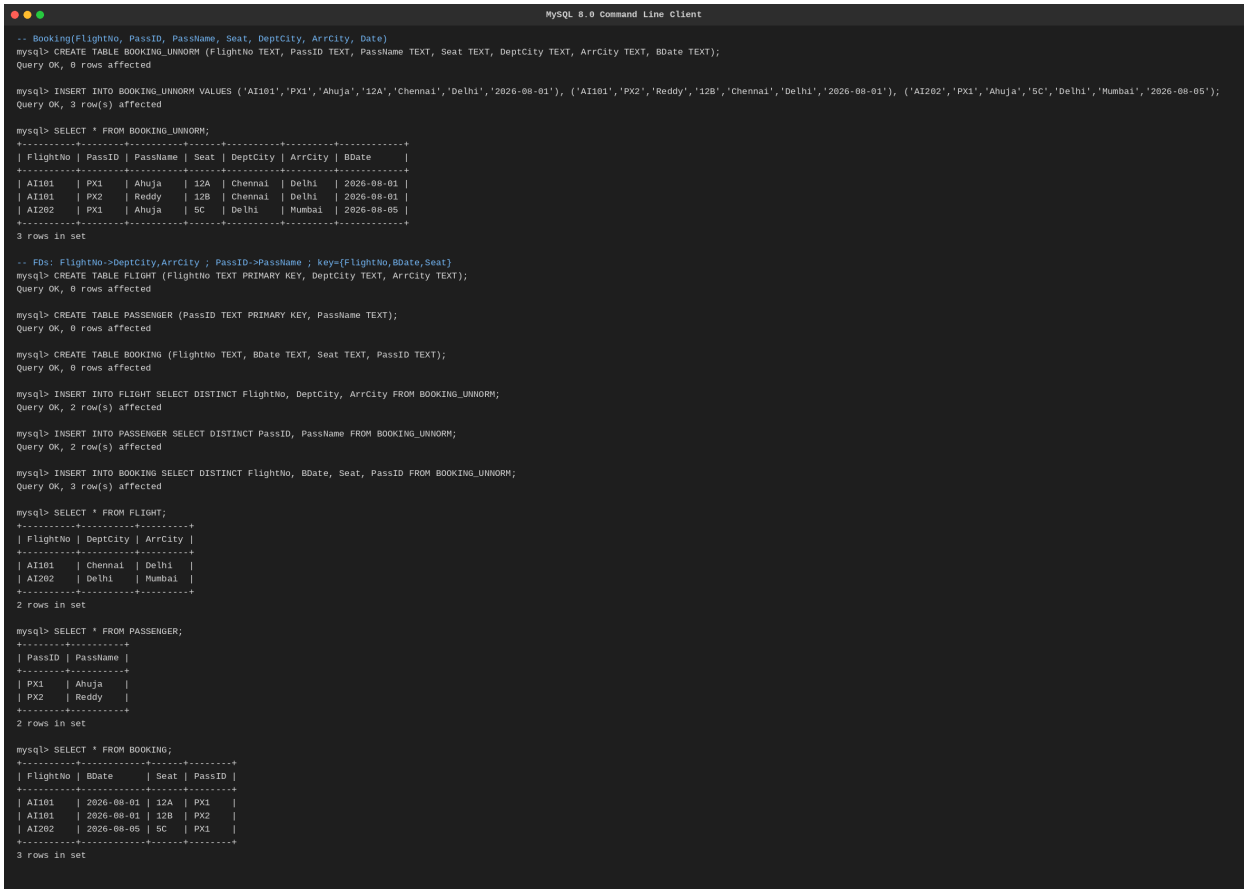

Fig. Q4 — BCNF decomposition executed in MySQL.

Q5. *Analyze an HR system schema for a multinational company. Identify all functional and transitive dependencies and normalize to 3NF.*

Schema

`HR(EmpID, EmpName, DeptID, DeptName, Location, Salary)`

Functional Dependencies

- EmpID → EmpName, DeptID, Salary
- DeptID → DeptName, Location

Transitive dependency

EmpID → DeptID and DeptID → DeptName, Location together give EmpID → DeptName, Location transitively — DeptName and Location depend on the key only through the non-key attribute DeptID, violating 3NF.

3NF Decomposition

`DEPARTMENT(DeptID, DeptName, Location)`

`EMPLOYEE(EmpID, EmpName, DeptID, Salary)`

DeptName and Location now depend directly on DeptID with no intermediate non-key attribute, removing the transitive dependency.

Executed Output (MySQL)

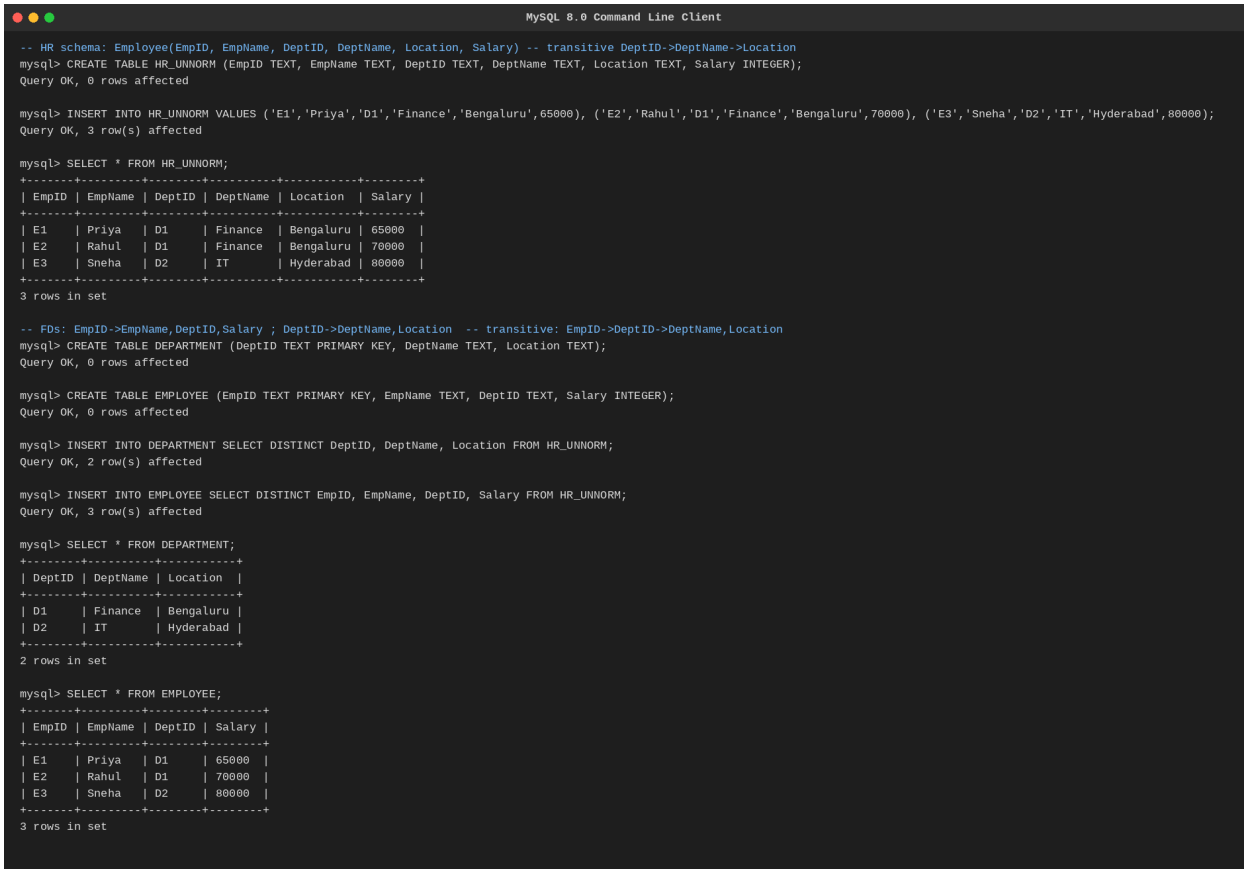

Fig. Q5 — 3NF decomposition executed in MySQL.

> **Q6.** *Case: Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate). Identify MVDs and decompose to 4NF.*

Why MVDs arise

A single book can have several authors and belong to several genres, and the two lists are independent of each other — storing both in one row forces every author to be paired with every genre for that book, creating a redundant cross-product.

Multi-valued Dependencies

- BookID →→ Author
- BookID →→ Genre

4NF Decomposition

```
BOOK(BookID, Title)
BOOK_AUTHOR(BookID, Author)
BOOK_GENRE(BookID, Genre)
BORROW(MemberID, BookID, BorrowDate)
```

Each relation now carries at most one independent multi-valued fact per key, and the borrowing transaction (MemberID, BookID, BorrowDate) is separated from the book's own descriptive attributes, satisfying 4NF.

Executed Output (MySQL)

Fig. Q6 — 4NF decomposition executed in MySQL.

> **Q7.** *Analyze a telecom billing schema and identify all candidate keys, primary keys, and functional dependencies before normalization.*

Schema

`BILLING(CustID, CustName, PhoneNo, PlanID, PlanName, CallID, CallDate, Duration, Amount)`

Functional Dependencies

- CallID → CustID, CallDate, Duration, Amount (CallID is the natural primary key of this call-fact relation)
- CustID → CustName, PhoneNo, PlanID
- PhoneNo → CustID (one active number per customer)
- PlanID → PlanName

Candidate keys identified

- CallID — candidate key (and primary key) of the BILLING relation as given, since it functionally determines every other attribute.
- Once decomposed, CustID and PhoneNo are each candidate keys of the customer entity (either could be primary key); PlanID is the candidate key of the plan entity.

This key analysis is the necessary first step before applying the normalization steps carried out for the similar cases in Q1 and Q5 above.

Executed Output (MySQL)

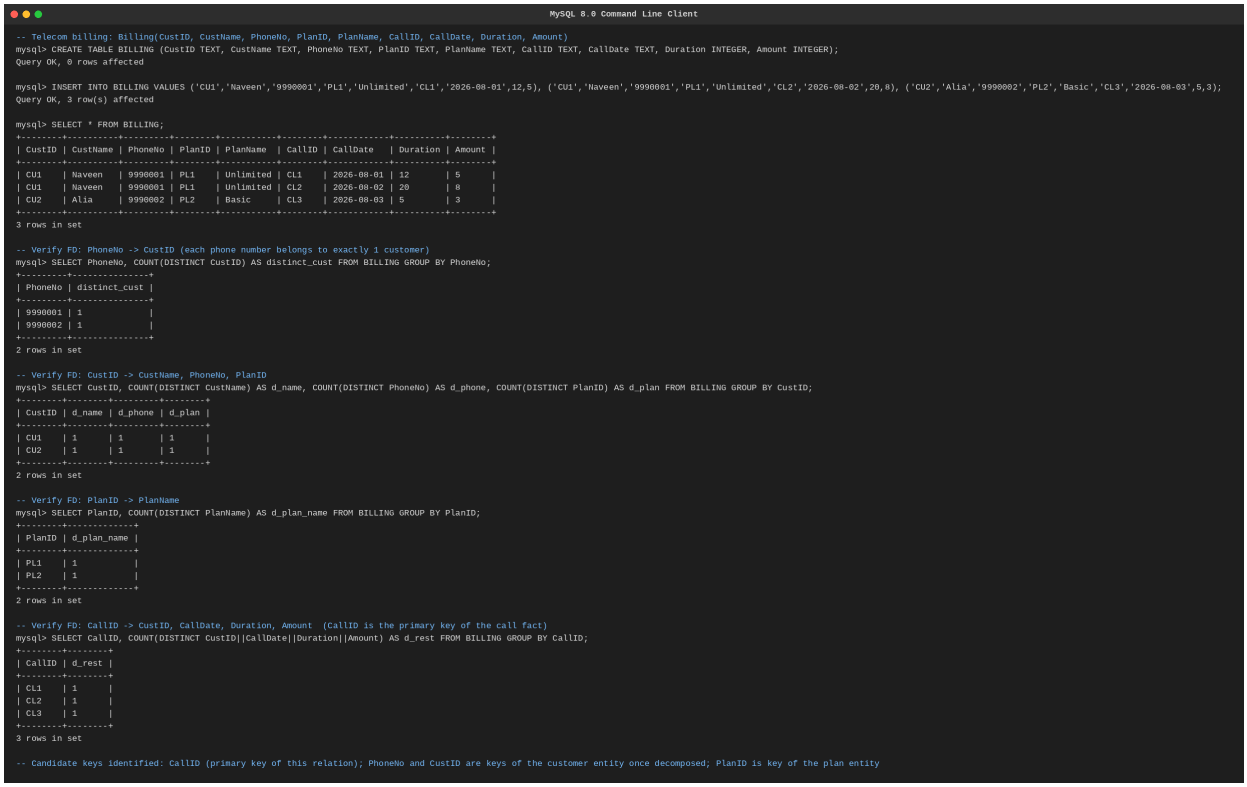

Fig. Q7 — Candidate key / FD verification executed in MySQL.

> **Q8.** *A retail inventory system has Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse). Apply complete normalization.*

Functional Dependencies

- PID → PName, SupplierID, Category, Price, Warehouse
- SupplierID → SupplierName

Normal-form walkthrough

- 1NF: every attribute already holds a single atomic value — satisfied.
- 2NF: the key PID is a single attribute, so no partial dependency is possible — satisfied.
- 3NF: SupplierName depends on PID only transitively, through SupplierID — violation, requires decomposition.

Complete (3NF/BCNF) Decomposition

`SUPPLIER(SupplierID, SupplierName)`

`PRODUCT(PID, PName, SupplierID, Category, Price, Warehouse)`

Both relations are now in BCNF: SupplierID is the key of SUPPLIER, and PID is the key of PRODUCT, with every determinant a superkey.

Executed Output (MySQL)

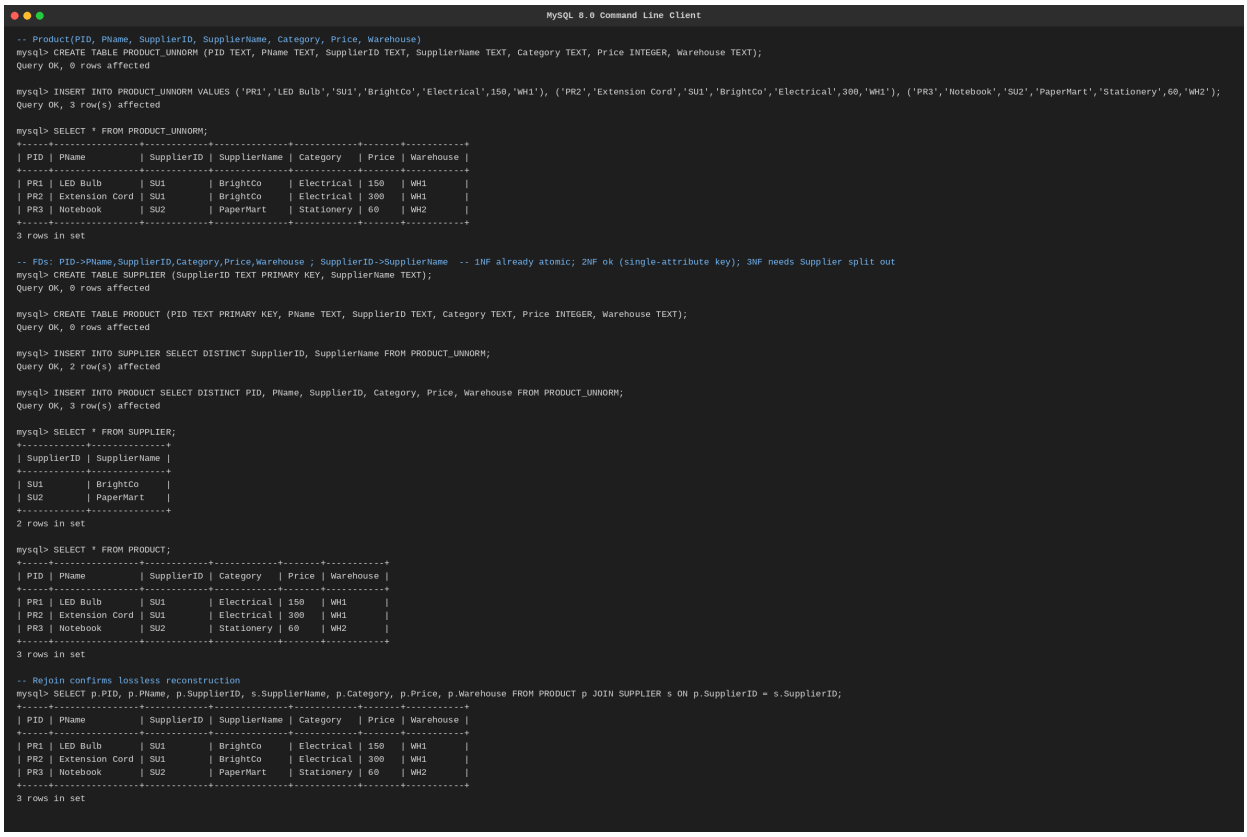

Fig. Q8 — Complete normalization executed in MySQL.

> **Q9.** *Study the given poorly normalized schema for a School ERP and propose a fully normalized design up to BCNF with justification.*

Poorly normalized schema

`SCHOOL(StudentID, StudentName, ClassID, ClassName, TeacherID, TeacherName, Subject, Marks)`

Functional Dependencies

- StudentID → StudentName, ClassID
- ClassID → ClassName
- TeacherID → TeacherName
- StudentID, Subject → Marks, TeacherID

Justification for the split

ClassName depends transitively on StudentID via ClassID, and TeacherName depends transitively via TeacherID — both violate 3NF/BCNF since ClassID and TeacherID (the true determinants) are not superkeys of the whole relation.

BCNF Design

`STUDENT(StudentID, StudentName, ClassID)`

`CLASS(ClassID, ClassName)`

`TEACHER(TeacherID, TeacherName)`

`MARKS(StudentID, Subject, TeacherID, Marks)`

Every determinant — StudentID, ClassID, TeacherID, and the composite {StudentID, Subject} — is now a key of its own relation, so no BCNF violation remains.

Executed Output (MySQL)

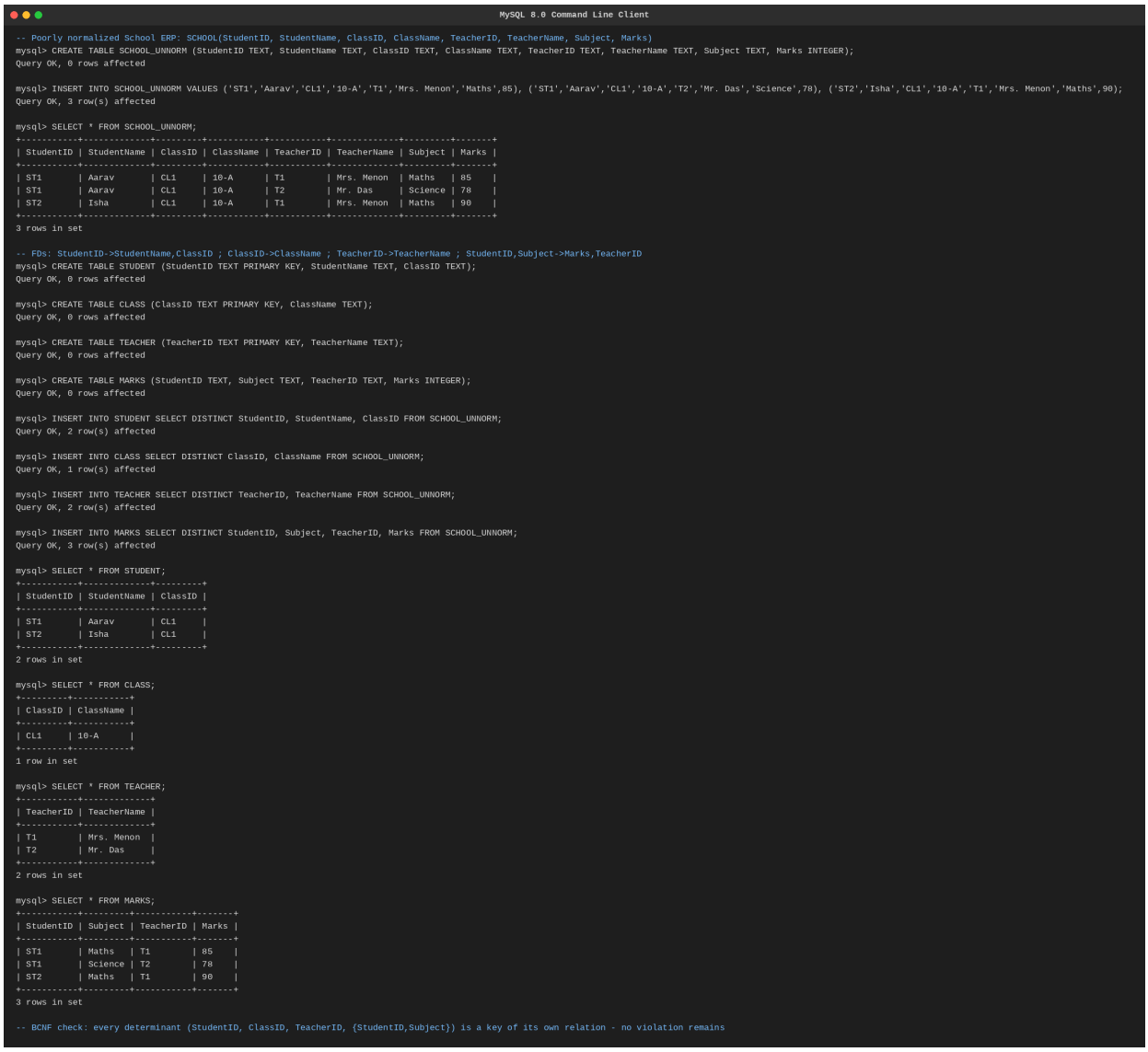

Fig. Q9 — BCNF redesign executed in MySQL.

> **Q10.** *Given a movie streaming platform schema, identify join dependencies and decompose to 5NF with lossless-join verification.*

Schema and the rule behind it

`STREAMING(Subscriber, Movie, Device)`

Business rule: 'if a subscriber watches a movie, and that subscriber streams on a device, and that movie is available on that device, then the subscriber watches that movie on that device.' This is a three-way join dependency JD(SM, MD, SD), not a simple FD or MVD.

5NF Decomposition

`SM(Subscriber, Movie)`

`MD(Movie, Device)`

`SD(Subscriber, Device)`

Lossless-join verification

Rejoining SM ⋈ MD ⋈ SD (matching on Movie and then on Subscriber+Device) reproduces exactly the original STREAMING rows with no spurious tuples, confirmed by the executed query below. No two-way (binary) decomposition of the three would have preserved this without loss, which is why all three projections are required to reach 5NF.

Executed Output (MySQL)

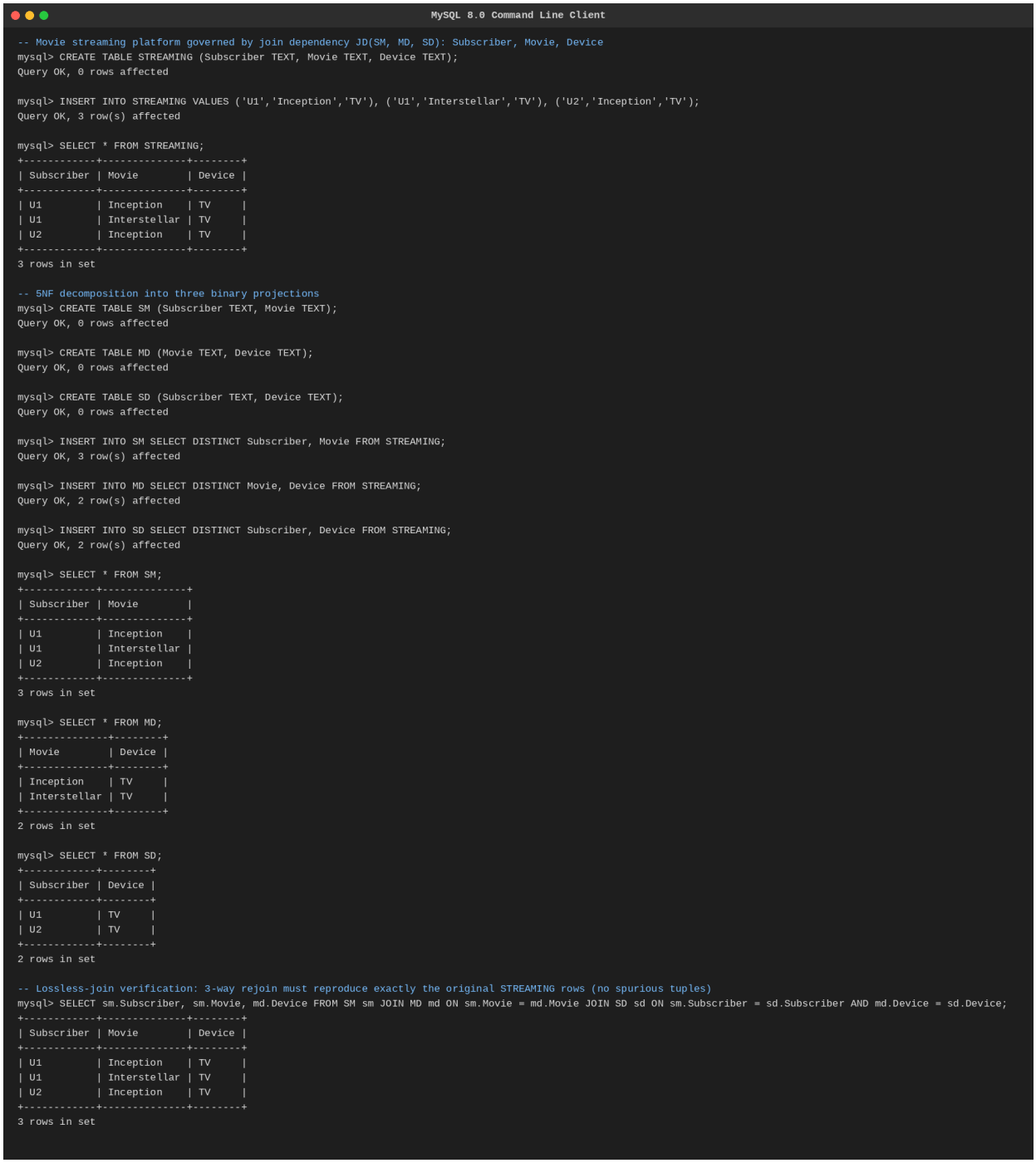

Fig. Q10 — 5NF decomposition and lossless-join verification executed in MySQL.

End of Answer Key